سرور کوچک تراکنش‌های بانکی (Mini Banking Transaction Server)

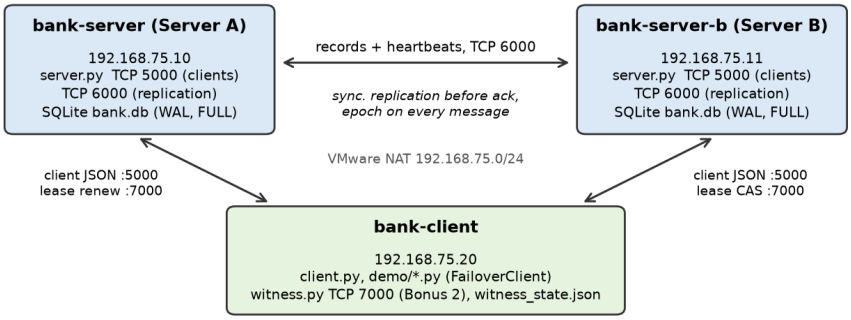
متین قنبری — درس سیستم‌های توزیع‌شده، تمرین ۱ (به‌همراه بخش امتیازی ۱ و ۲)

مخزن کد: https://github.com/MatinGhanbari/distributed-banking-assignment

کامیت‌ها: `ccb4e90` بخش اصلی، `4bc710f` امتیازی ۱، `9a79f27` امتیازی ۲

۱. معماری و استقرار

سامانه با Python 3.12 و فقط با کتابخانه‌ی استاندارد نوشته شده است (هیچ بسته‌ی جانبی لازم نیست و `requirements.txt` همین را مستند می‌کند). استقرار روی سه ماشین مجازی VMware با Ubuntu 24.04.3 در شبکه‌ی NAT با نشانی 192.168.75.0/24 است: **bank-server** (سرور A، 192.168.75.10)، **bank-server-b** (سرور B، 192.168.75.11) و **bank-client** (192.168.75.20) که کلاینت، اسکریپت‌های نمایشی و در بخش امتیازی ۲ فرایند witness (گواه) را اجرا می‌کند. درگاه 5000 برای کلاینت‌ها، درگاه 6000 برای تکثیر (replication) میان دو سرور و درگاه 7000 برای witness است. کلاینت همیشه به IP واقعی سرور وصل می‌شود، بنابراین در لاگ سرور مقدار `client_ip` برابر 192.168.75.20 دیده می‌شود. بدون `--peer` سرور همان سرور تکی بخش اصلی است؛ `--peer` امتیازی ۱ و `--witness` امتیازی ۲ را فعال می‌کند.



شکل ۱. سه ماشین مجازی، نقش‌ها و درگاه‌ها.

۲. روش ارتباط و قالب پیام

روش ارتباطی **سوکت خام TCP** است (`socket/socketserver`). TCP خودش جریان بایتیِ مطمئن و مرتب فراهم می‌کند، پس برنامه فقط باید مرز پیام‌ها را تعیین کند: هر درخواست و هر پاسخ **دقیقاً یک شیء JSON در یک خط** با کدگذاری UTF-8 است (newline-delimited JSON). کلاینت می‌تواند روی یک اتصال چند درخواست بفرستد و پاسخ‌ها به همان ترتیب برمی‌گردند. فیلدها: `action` (یکی از `transfer`، `withdraw`، `deposit`، `balance`)، `account`، `amount` هم لازم است) و `to_account` (در انتقال، `request_id` اختیاری؛ اگر شناسه فرستاده نشود سرور خودش `srv-…` می‌سازد و این شناسه در پاسخ و لاگ تکرار می‌شود. مبلغ‌ها با `Decimal` (و نه float) خوانده می‌شوند، باید بزرگ‌تر از صفر، حداکثر با دو رقم اعشار و حداکثر 1,000,000,000 باشند و به **سنت صحیح** (integer cents) تبدیل می‌شوند.

```
-> {"request_id": "r1", "action": "withdraw", "account": "A1001", "amount": 30}
<- {"status": "success", "request_id": "r1", "action": "withdraw", "account": "A1001",
    "balance": 970.0, "balance_cents": 97000, "amount": 30.0}
<- {"status": "error", "request_id": "r2", "code": "INSUFFICIENT_FUNDS",
    "message": "insufficient funds in 'A1002'", "account": "A1002", "balance": 500.0,
    "balance_cents": 50000}
```

در حالت خوشه‌ای، پاسخ موفق فیلدهای `served_by` و `epoch` را هم دارد. پس از هر خطا، حتی بایت‌های تصادفی، اتصال برای خط بعدی باز می‌ماند و باقی‌مانده‌ی خطِ بلندتر از 64 KB دور ریخته می‌شود. چون `transfer` پیاده‌سازی شده است، حالت «عمل پشتیبانی‌نشده» با عملی مانند `"steal"` نشان داده می‌شود.

کد خطا	شرایط	کد خطا	شرایط
`UNKNOWN_ACCOUNT`	حساب یا حساب مقصد وجود ندارد (مثلاً Z9999)	`SAME_ACCOUNT`	انتقال به همان حساب
`INSUFFICIENT_FUNDS`	برداشت/انتقال بیش از موجودی	`DUPLICATE_REQUEST_ID`	استفاده‌ی دوباره از یک request_id برای عملی دیگر
`INVALID_AMOUNT`	مقادیر -50، 0، "abc"، true، NaN، Infinity، 10.005 یا بیش از حد	`SERVER_BUSY`	هنگام اتصال، وقتی max_clients اتصال فعال است؛ سپس اتصال بسته می‌شود
`UNSUPPORTED_ACTION`	action جزو چهار عمل بالا نیست	`FAULT_INJECTION_DISABLED`	پرچم simulate_failure/crash بدون فعال بودن حالت اشکال‌زدایی
`MISSING_FIELD`	نبودِ amount، account، action یا to_account	`INTERNAL_ERROR`	استثنای پیش‌بینی‌نشده؛ تراکنش برگردانده شد
`MALFORMED_REQUEST`	JSON خراب، غیرشیء، غیر UTF-8، خط بیش از 64 KB، نوع نادرست فیلد	`NOT_PRIMARY` / `NO_QUORUM`	امتیازی: گره پشتیبان است / lease ندارد؛ هر دو primary_hint دارند

۳. کلاینت‌های هم‌زمان و وضعیت مشترک

سرور یک `socketserver.ThreadingTCPServer` است: حلقه‌ی پذیرش، هر اتصال را به یک **نخ (thread) جدید** می‌دهد که تا قطع اتصال به همان کلاینت سرویس می‌دهد؛ بنابراین کلاینت کُند بقیه را معطل نمی‌کند. سقف `max_clients` قابل پیکربندی است و اولویت آن چنین است: **خط فرمان** `--max-clients` **بالاتر از متغیر محیطی** `BANK_MAX_CLIENTS` **بالاتر از** `config.json` **و در نهایت مقدار پیش‌فرض 100**. بررسی سقف و ثبت سوکت جدید زیر یک قفل در `process_request` انجام می‌شود، پس شرایط مسابقه نمی‌تواند سقف را بشکند؛ کلاینت اضافی پیام `SERVER_BUSY` می‌گیرد، اتصالش بسته می‌شود و رویداد `rejected_busy` ثبت می‌شود. مهلت بیکاری 300 ثانیه‌ای جای کلاینت‌های مرده را آزاد می‌کند و با SIGINT/SIGTERM سرور به‌آرامی خاموش می‌شود (درخواست‌های در جریان یا commit می‌شوند یا برمی‌گردند).

همه‌ی حساب‌ها **وضعیت مشترکی در یک فایل پایگاه‌داده‌ی SQLite** هستند که همه‌ی نخ‌ها روی آن کار می‌کنند؛ هر نخ اتصال **اختصاصی** خودش را باز می‌کند (اتصال‌ها هرگز میان نخ‌ها مشترک نیستند) با `journal_mode=WAL` (WAL یعنی write-ahead log: تغییرات ابتدا در یک لاگ پیش‌نویس نوشته می‌شوند)، `synchronous=FULL` و مهلت انتظار 30 ثانیه برای قفل. جدول‌ها: `accounts(account_id PRIMARY KEY, balance_cents INTEGER NOT NULL CHECK (balance_cents >= 0))`، جدول `processed_requests(request_id PRIMARY KEY, fingerprint, response_json, created_at)` برای idempotency (اجرای تنها یک‌باره‌ی درخواست تکراری) و جدول `node_state(key, value)` برای نقش، epoch و `last_lsn` در خوشه. حساب‌های ساختگی A1001 = 1000.00, A1002 = 500.00, A1003 = 2500.00 وقتی پایگاه‌داده خالی است ساخته می‌شوند. استفاده از موتور تراکنشی به‌جای dict و قفل، اتمیک بودن، جداسازی و ماندگاری در برابر خرابی را از کدی آزموده‌شده به دست می‌دهد.

۴. درستی عملیات هم‌زمان روی یک حساب

خطای کلاسیک، به‌روزرسانی گم‌شده یا برداشت بیش از موجودی است (دو نخ هر دو 1000 را می‌خوانند، هر دو 30 را مجاز می‌دانند و هر دو می‌نویسند). این خطا به دو روش جلوگیری می‌شود: (۱) هر تغییر موجودی درون `BEGIN IMMEDIATE … COMMIT` اجرا می‌شود؛ IMMEDIATE قفل نوشتن SQLite را در **ابتدای** تراکنش می‌گیرد، پس نویسنده‌ها پشت سر هم اجرا می‌شوند و دومی نتیجه‌ی commitشده‌ی اولی را می‌بیند؛ (۲) برداشت یک **به‌روزرسانی شرطی** است: `UPDATE accounts SET balance_cents = balance_cents - ? WHERE account_id = ? AND balance_cents >= ?`؛ پس «بررسی موجودی» و «کم کردن» یک دستور واحدند و اگر هیچ سطری تغییر نکند پاسخ `INSUFFICIENT_FUNDS` است. در نتیجه هر مجموعه از عملیات موازی به حالتی می‌رسد که با اجرای پشت‌سرهم آن‌ها نیز قابل دستیابی است. خواندن `balance` یک SELECT تکی است که در حالت WAL آخرین وضعیت commitشده را می‌بیند و نویسنده‌ها را مسدود نمی‌کند. **شاهد:** 50 کلاینت با اتصال‌های جداگانه که با `threading.Barrier` هم‌زمان رها می‌شوند، هر کدام 30.00 از A1001 (با موجودی 1000.00) برداشت می‌کنند: **33 موفق، 17 با INSUFFICIENT_FUNDS و موجودی نهایی 10.00** (یعنی 30 × 33 - 1000) و هرگز منفی نمی‌شود.

۵. تضمین هر یک از ویژگی‌های ACID

ACID چهار ویژگی تراکنش است: اتمیک بودن (Atomicity)، سازگاری (Consistency)، جداسازی (Isolation) و ماندگاری (Durability).

اتمیک بودن (Atomicity). متد `Bank._transaction()` هر عمل تغییردهنده را در بر می‌گیرد: همه‌ی گام‌ها (برداشت، واریز، سطر `processed_requests` و در حالت خوشه گام تکثیر) درون یک تراکنش‌اند و **هر** استثنا به `ROLLBACK` می‌انجامد؛ پس برداشت و واریزِ یک انتقال یا با هم انجام می‌شوند یا هیچ‌کدام. اگر فرایند وسط تراکنش با `kill -9` کشته شود، فقط قاب‌های commitنشده در WAL می‌مانند که SQLite در اجرای بعدی نادیده می‌گیرد. **نمایش** (`fault_injection_demo.py` با سروری که با `BANK_ENABLE_FAULT_INJECTION=1` اجرا شده): یک انتقال از A1001 به A1002 و یک برداشت با `"simulate_failure": true` فرستاده می‌شوند؛ سرور درست پس از برداشت استثنا می‌دهد، کلاینت `INTERNAL_ERROR` می‌گیرد و هر دو موجودی بدون تغییر می‌مانند (در اجرای روی VMها 230.70 و 805.36). بدون این پرچم اشکال‌زدایی همان درخواست پاسخ `FAULT_INJECTION_DISABLED` می‌گیرد، پس این پرچم در کار عادی هرگز پولی جابه‌جا نمی‌کند.

سازگاری (Consistency). ورودی پیش از هر کار با پایگاه‌داده اعتبارسنجی می‌شود (عمل معتبر، مبلغ مثبت با حداکثر دو رقم اعشار، وجود حساب‌ها، انتقال نکردن به خود)؛ سنت صحیح خطای گرد کردن را حذف می‌کند؛ به‌روزرسانی شرطی جلوی برداشت اضافی را می‌گیرد و قید `CHECK (balance_cents >= 0)` در طرح جدول آخرین خط دفاع است. انتقال پول را جابه‌جا می‌کند ولی پول نمی‌سازد. **نمایش:** `multi_clients.py` (8 کلاینت × 25 عمل تصادفی واریز/برداشت/انتقال/خواندن): 189 عمل موفق، 11 عمل رد شد و مجموع پول برابر «قبلی + واریزها - برداشت‌ها» ماند (**4577.63**)؛ `error_cases.py` نشان می‌دهد هر 17 ورودی نامعتبر بدون هیچ تغییری در وضعیت رد می‌شوند.

جداسازی (Isolation). `BEGIN IMMEDIATE` همه‌ی نویسنده‌ها را پشت سر هم اجرا می‌کند که برای نوشتن‌ها معادل جداسازی serializable است؛ خواننده‌ها در WAL فقط داده‌ی commitشده را می‌بینند، پس خواندن کثیف (dirty read) و به‌روزرسانی گم‌شده رخ نمی‌دهد. جست‌وجوی idempotency هم درون همان تراکنشِ قفل‌شده است، پس دو تلاش هم‌زمانِ یک درخواست نمی‌توانند هر دو اعمال شوند. **نمایش:** همان آزمون مسابقه‌ی 30 × 50 در بخش ۴.

ماندگاری (Durability). در حالت WAL با SQLite، `synchronous=FULL` در هر COMMIT فایل WAL را fsync می‌کند و سرور پیام `success` را **فقط پس از بازگشت COMMIT** می‌فرستد. **نمایش** (`durability_demo.py`): پنج واریز 100 تأییدشده به A1003،

سپس `kill -9` سرور (بدون هیچ پاک‌سازی) و اجرای دوباره؛ موجودی **4541.57** باقی ماند. **Idempotency** این را کامل می‌کند: کلاینتی که پاسخ را از دست داده همان `request_id` را دوباره می‌فرستد و به‌جای اعمال دوباره، پاسخ ذخیره‌شده را با `"replayed": true` می‌گیرد؛ استفاده‌ی دوباره از یک شناسه برای عملی دیگر `DUPLICATE_REQUEST_ID` می‌دهد (فقط عملیات موفق ثبت می‌شوند).

۶. ثبت رویدادها (Logging)

هر درخواست و پاسخ یک خط JSON است که ماژول `logging` (ایمن در برابر نخ‌ها) هم در stdout و هم در `logs/server.log` می‌نویسد. فیلدها: `timestamp` (ISO-8601 به وقت UTC با میلی‌ثانیه)، `request_id`، `client_ip`، `client_port`، `action`، `account`، `to_account`، `amount`، `result` (`success` یا کد خطا)، `balance`، `replayed` و `latency_ms` (و در خوشه `node_id` و `epoch`). اتصال، قطع اتصال (با دلیل)، `internal_error`، `rejected_busy`، سیگنال‌ها و شروع/توقف هم ثبت می‌شوند، به‌علاوه‌ی رویدادهای خوشه (`promotion_refused`، `lease_restored`، `lease_lost`، `lease_granted`، `role_change`، `promoted`)؛ witness در `logs/witness.log` می‌نویسد. نمونه (فیلدها کوتاه شده‌اند):

```
{"timestamp": "2026-10-21T15:35:25.245Z", "event": "request", "request_id": "cli-1a2b",
 "client_ip": "192.168.75.20", "client_port": 51544, "action": "withdraw", "account": "A1001",
 "result": "success", "balance": 970.0, "latency_ms": 3.1, ...}
```

۷. امتیازی ۱ – خوشه‌ی اصلی/پشتیبان با failover خودکار

failover یعنی ادامه‌ی خودکار سرویس روی سرور پشتیبان پس از خرابی سرور اصلی (primary). **ترتیب نوشتن: تکثیر پیش از تأیید.** برای هر واریز/برداشت/انتقال، سرور اصلی (۱) `BEGIN IMMEDIATE` را اجرا، اعتبارسنجی و موجودی‌ها را به‌روز می‌کند و سطر `processed_requests` را درج می‌کند؛ (۲) در همان تراکنش `lsn = last_lsn + 1` (شماره‌ی ترتیب لاگ) را برمی‌دارد؛ (۳) رکورد `{lsn, epoch, request_id, balances, processed}` را با **موجودی‌های مطلق جدید** روی درگاه 6000 می‌فرستد و **منتظر تأیید می‌ماند**؛ (۴) پشتیبان آن را در **یک** تراکنش محلی اعمال می‌کند و فقط **پس از COMMIT خودش** تأیید می‌دهد؛ (۵) تنها پس از آن سرور اصلی COMMIT می‌کند و پاسخ می‌دهد. قفل نوشتن تضمین می‌کند رکوردها به ترتیب lsn بروند؛ پشتیبان رکورد n را فقط وقتی اعمال می‌کند که `last_lsn = n-1` باشد، رکورد تکراری را بدون اعمال دوباره تأیید می‌کند و در صورت شکاف پاسخ `NEED_SNAPSHOT` می‌دهد. وقتی پشتیبان (دوباره) وصل می‌شود، اصلی **با نگه داشتن قفل نوشتن یک snapshot کامل** می‌فرستد و فقط پس از تأیید آن، پشتیبان «هم‌گام» (in sync) است. اگر پشتیبان در دسترس نباشد (`repl_timeout = 2 s`)، اصلی `degraded_mode` را ثبت و به‌تنهایی ادامه می‌دهد. بررسی روی VM: پس از یک واریز روی A، سرور B رویداد `replica_applied` را در 16:23:30.038 UTC ثبت کرد، پیش از پاسخ A در 16:23:30.042؛ درخواست مستقیم به B پاسخ `NOT_PRIMARY` همراه `primary_hint` می‌دهد.

ضربان (heartbeat)، مهلت و epoch. اصلی هر 0.5 ثانیه یک heartbeat می‌فرستد. پشتیبانی که 3 ثانیه (`failover_timeout`) چیزی نشنود **و هم‌گام باشد** خودش را ارتقا می‌دهد: `epoch += 1` و `role = primary` هر دو در `node_state` ذخیره و سپس به کلاینت‌ها سرویس می‌دهد؛ پشتیبانی که هرگز snapshot نگرفته `cannot_promote` ثبت می‌کند. epoch شماره‌ی دوره‌ی رهبری است و هر پیام میان سرورها epoch فرستنده را دارد؛ پیام با epoch کمتر با `STALE_EPOCH` رد می‌شود و اصلی‌ای که `STALE_EPOCH` بگیرد یا اصلیِ دارای epoch بالاتر ببیند **بی‌درنگ کنار می‌رود** و پشتیبان می‌شود.

تلاش دوباره‌ی کلاینت و بازگشت گره. `client.FailoverClient` نشانی هر دو سرور را می‌داند و برای هر درخواست یک `request_id` می‌سازد. در صورت خطای اتصال، پایان مهلت، `SERVER_BUSY`، `NOT_PRIMARY` یا `NO_QUORUM`، **همان بایت‌ها با همان شناسه** را با تأخیر کوچک فزاینده (حداکثر 1 ثانیه) به سرور دیگر می‌فرستد تا پاسخی برسد یا 30 ثانیه بگذرد. گره‌ای که دوباره راه می‌افتد ابتدا به همتایش `hello` می‌فرستد؛ اگر همتا اصلی با epoch بزرگ‌تر یا مساوی باشد، پشتیبان می‌شود و پیش از هر ارتقا باید snapshot بگیرد؛ پس A پس از بازگشت هرگز اصلیِ دوم نمی‌شود.

چرا هیچ تراکنشی گم یا دو بار اعمال نمی‌شود. گم نشدن: هر نوشتن فقط پس از commit روی B تأیید می‌شود و B فقط در حالت هم‌گام ارتقا می‌یابد، پس هنگام مرگ A هر نوشتن تأییدشده روی B هست. دو بار اعمال نشدن: سطر `processed_requests` در همان رکورد و همان تراکنش پشتیبان همراه تغییر موجودی است، پس B یا هر دو را دارد یا هیچ‌کدام؛ بنابراین تلاش دوباره یا نخستین اعمال است یا بازپخش پاسخ ذخیره‌شده (`"replayed": true`). درخواستی که A آن را commit نکرده روی A برگردانده می‌شود؛ اگر به B رسیده بود انجام‌شده حساب می‌شود و A هنگام بازگشت با snapshot آن را می‌گیرد، وگرنه هرگز رخ نداده است. رکوردها موجودی مطلق را به ترتیب lsn دارند، پس B دقیقاً وضعیت A را بازمی‌سازد. یک آزمون واحد خوشه سخت‌ترین حالت را با تزریق خطا می‌سازد: A پس از تأیید B و پیش از COMMIT خودش خارج می‌شود؛ تلاش دوباره‌ی کلاینت روی B پاسخ `"replayed": true` می‌گیرد و پول دقیقاً یک بار جابه‌جا می‌شود.

۸. امتیازی ۲ – جلوگیری از split brain با witness و lease

split brain (مغز دوپاره) یعنی دو سرور هم‌زمان خود را اصلی بدانند و داده‌ها واگرا شوند. با دو گره، B نمی‌تواند «A از کار افتاده» را از «پیوند A–B قطع شده» تشخیص دهد و در امتیازی ۱ هنگام partition هر دو سرویس می‌دهند. امتیازی ۲ یک رأی سوم اضافه می‌کند: `witness.py` (گواه/داور) روی ماشین کلاینت و درگاه 7000. witness هیچ داده‌ی بانکی ندارد و فقط `{epoch, holder, in_sync, sync_seq, ttl}` را نگه می‌دارد و یک عمل دارد: `acquire`. **دارنده‌ی فعلی تمدید می‌کند** (انقضا = اکنون + TTL و `sync_seq` جدیدتر مقدار `in_sync` را به‌روز می‌کند). **هر گره دیگر** فقط با **compare-and-set** ارتقا می‌یابد: وقتی now > expiry +

(grace = 1 s)، آخرین گزارش دارنده‌ی قبلی گفته باشد پشتیبان هم‌گام است و epoch درخواست‌کننده کهنه نباشد؛ آنگاه epoch := max(epoch, caller) + 1. درخواست‌ها زیر یک قفل پردازش می‌شوند و هر تغییر **پیش از پاسخ** در `witness_state.json` نوشته می‌شود (فایل موقت + fsync + rename + fsync پوشه). witness پس از راه‌اندازی دوباره، محافظه‌کارانه lease دارنده را تا یک TTL دیگر معتبر می‌داند.

lease با مهلت محافظه‌کارانه. lease مجوز زمان‌دار سرویس‌دهی است. اصلی هر 1 ثانیه با TTL = 3 s تمدید می‌کند. مهلت محلی آن **زمان ارسال تمدید + TTL − margin** است (`t_send + 3 − 0.5`)؛ انقضای witness برابر `t_receive + 3 ≥ t_send + 3` است و فقط پس از **انقضا + grace** lease را به گره دیگری می‌دهد. پس اصلی قبلی دست‌کم margin + grace = 1.5 s پیش از آنکه دیگری اجازه‌ی شروع داشته باشد متوقف می‌شود. `check_can_serve()` پیش از **هر** درخواست، حتی خواندن، اجرا می‌شود (گره حصارکشی‌شده ممکن است داده‌ی کهنه داشته باشد) و lease پس از تراکنش دوباره بررسی می‌شود؛ بدون lease، گره `NO_QUORUM` پاسخ می‌دهد و `lease_lost` ثبت می‌کند. heartbeatها فیلد `lease: true/false` دارند و heartbeat اصلیِ بدون lease «زنده» حساب نمی‌شود. سرور اجرا نمی‌شود مگر آنکه `lease_renew + witness_timeout < lease_ttl − lease_margin` برقرار باشد (2.5 > 1 + 1).

قاعده‌ی هم‌گامی و حصارکشی با epoch (fencing). در حالت degraded (قطع پیوند با B)، A فقط **پس از تأیید گزارش «ناهم‌گام» توسط witness** می‌تواند نوشتن را commit کند، وگرنه نوشتن با `NO_QUORUM` برگردانده می‌شود. چون ارتقا نیازمند in_sync = true است، B هرگز بدون نوشتن‌های تأییدشده‌ی A جای آن را نمی‌گیرد. epochها را فقط witness صادر می‌کند؛ رکورد، heartbeat یا snapshot با epoch قدیمی‌تر رد می‌شود (`STALE_EPOCH`) و هر گره‌ای که epoch بالاتری ببیند کنار می‌رود؛ گره در حال راه‌اندازی ابتدا از witness می‌پرسد و اگر دیگری lease دارد، پشتیبان شروع می‌کند. **حالت‌های partition:** قطع A–B در حالی که هر دو به witness می‌رسند ← A lease را نگه می‌دارد و degraded سرویس می‌دهد و تلاش‌های B با `promotion_refused` رد می‌شود؛ جدا شدن A از B و از A ← witness در مهلت خود را حصار می‌کند، B پس از انقضا + grace lease (با 1 + epoch) می‌گیرد و پس از وصل شدن، A کنار می‌رود و snapshot B را می‌گیرد؛ از کار افتادن witness ← هیچ خواندن و نوشتنی در هیچ‌جا انجام نمی‌شود.

مصالحه‌ی CP و فرض‌های ساعت. طراحی آگاهانه **CP** است (در قضیه‌ی CAP، سازگاری بر دسترس‌پذیری مقدم است): هنگام partition، سمت اقلیت دسترس‌پذیری را از دست می‌دهد (و وقتی witness خاموش است کل خوشه) تا خطر دو اصلی پیش نیاید؛ هیچ کلید «fail-open» وجود ندارد. فقط **مدت‌زمان‌هایی که هر ماشین با ساعت یکنوای (monotonic) خودش می‌سنجد** به کار می‌رود و هرگز ساعت دیواری دو ماشین مقایسه نمی‌شود؛ پس پرش NTP و اختلاف ساعت اثری ندارد. فرض‌ها: **رانش محدود ساعت** (وقتی witness 3 s + 1 s می‌شمارد، ساعت A نباید کمتر از 2.5 s بشمارد؛ یعنی حدود 30٪ اختلاف نرخ تحمل می‌شود) و **توقف نکردن فرایند** بیش از فاصله‌ی میان بررسی lease و خروج پاسخ (هشدار کلاسیک lease). استدلال ایمنی داده (گم نشدن نوشتن تأییدشده و واگرا نشدن) به ساعت وابسته نیست: نوشتن‌های تکثیرشده به پذیرش B نیاز دارند و epochهای قدیمی را رد می‌کند؛ نوشتن‌های degraded به تأیید witness درباره‌ی عقب بودن B نیاز دارند.

۹. نتایج اندازه‌گیری‌شده روی ماشین‌های مجازی

نمایش / بررسی	نتیجه
concurrent_withdrawals (50 کلاینت × 30.00 روی A1001)	33 موفق، 17 INSUFFICIENT_FUNDS، موجودی نهایی 10.00 – PASS
multi_clients (8 کلاینت × 25 عمل)	189 موفق، 11 رد شده، پول با مقدار 4577.63 حفظ شد – PASS
fault_injection (انتقال + برداشت با simulate_failure)	بازگردانی کامل، موجودی‌ها بدون تغییر (230.70 و 805.36) – PASS
durability (kill -9 و اجرای دوباره)	موجودی 4541.57 باقی ماند – PASS
error_cases	هر 17 مورد PASS؛ اتصال پس از خطاها قابل استفاده ماند
max_clients (با `--max-clients 3`)	اتصال چهارم SERVER_BUSY گرفت؛ پس از بستن یک اتصال، کلاینت جدید پذیرفته شد
بررسی اولیه‌ی امتیازی ۱	رویداد replica_applied روی B در 16:23:30.038 UTC پیش از پاسخ A در 042.؛ درخواست مستقیم به B ← NOT_PRIMARY + primary_hint
failover امتیازی ۱ (6 کلاینت، 40 s، کشتن A در حدود ثانیه‌ی 12)	7824 پاسخ تأییدشده (7812 موفق، 12 INSUFFICIENT_FUNDS)؛ A به 2333 و B به 5491 پاسخ داد؛ 6 درخواست نیاز به تلاش دوباره داشتند و 0 بازپخش؛ بیشترین فاصله‌ی failover برابر 4.17 s؛ موجودی نهایی A1003 = 5098.73, A1002 = 2016.32, A1001 = 2974.91، همه برابر مقدار مورد انتظار؛ هر 4 بررسی PASS
بازگشت A در امتیازی ۱	A پس از اجرای دوباره role_change از primary به backup (همتای B اصلی با epoch 2) و snapshot_applied در lsn 7812 را ثبت کرد
امتیازی ۲، cut-link (فقط پیوند A و B)	دارنده‌ی lease در witness همان A ماند؛ B نتوانست ارتقا یابد؛ A به سرویس ادامه داد
امتیازی ۲، cut-all (جدا شدن A از B و از درگاه witness 7000)	آخرین تأیید A در 63.86 s و سپس 794 پاسخ NO_QUORUM؛ نخستین تأیید B در 67.41 s با epoch 2؛ فاصله‌ی 3.55 s بدون هم‌پوشانی

امتیازی ۲، پس از وصل دوباره / مجموع	A پشتیبان شد با digest یکسان 568b22e30e82c88b با B و 4916 نوشتن پردازش‌شده؛ 7101 پاسخ (4916 نوشتن موفق)؛ هر 6219 پاسخ A در 1 epoch و هر 882 پاسخ B در 2 epoch؛ هر 5 بررسی PASS
کنترل منفی (اجرای محلی، نه روی VMها)	`split_brain_demo.py --local --without-witness`: اصلی‌های هم‌پوشان و نوشتن‌های گم‌شده؛ یعنی بررسی‌ها split brain را تشخیص می‌دهند

۱۰. محدودیت‌ها

نوشتن‌ها پشت سر هم اجرا می‌شوند (یک قفل نوشتن SQLite و در خوشه به‌علاوه‌ی یک رفت‌وبرگشت شبکه و یک fsync روی B برای هر نوشتن)؛ این ساده و درست است ولی توان عملیاتی را محدود می‌کند. `synchronous=FULL` فقط وقتی در برابر قطع برق محافظت می‌کند که دیسک مجازی fsync را واقعاً اجرا کند. هیچ احراز هویت یا رمزنگاری روی هیچ درگاهی نیست (فقط شبکه‌ی آزمایشگاه). witness نقطه‌ی واحد **عدم دسترس‌پذیری** است (نه ناسازگاری)؛ سامانه‌ی واقعی از 3 تا 5 نسخه‌ی کامل با Raft/Paxos استفاده می‌کند. استدلال lease به رانش محدود ساعت و نبودِ توقف طولانی فرایند (مثلاً یخ‌زدن VM) وابسته است. اگر A در حالی بمیرد که B ناهم‌گام است، خوشه تا بازگشت A در دسترس نیست (طبق طراحی). نوشتنی که lease آن وسط درخواست تمام شود ممکن است commit شده ولی با `NO_QUORUM` پاسخ داده شود و کلاینت باید با همان شناسه دوباره بفرستد. failover حدود 3 تا 4 ثانیه طول می‌کشد، snapshot کل پایگاه‌داده را کپی می‌کند و `processed_requests` هرگز پاک نمی‌شود. بدون `--witness` (فقط امتیازی ۱) partition شبکه هنوز می‌تواند split brain بسازد، همان‌طور که کنترل منفی نشان می‌دهد.

۱۱. روش اجرا

پوشه‌ی پروژه را روی هر سه VM کپی کنید، `cd bank-assignment` را اجرا و `python3 --version` را بررسی کنید (نصب چیزی لازم نیست). اگر ufw فعال است، درگاه‌های 5000 و 6000 را روی سرورها و 7000 را روی ماشین کلاینت باز کنید. برای شروع تمیز: `rm -f bank.db bank.db-wal bank.db-shm` (و روی ماشین کلاینت `rm -f witness_state.json`). در فهرست زیر، پیشوند A یعنی سرور A، پیشوند B یعنی سرور B و پیشوند C یعنی ماشین کلاینت؛ همه‌ی دستورها از پوشه‌ی پروژه اجرا می‌شوند.

```
# ---- Part 1: server on A (192.168.75.10), client and demos on the client VM (192.168.75.20)
A: python3 server.py                    # listens on 0.0.0.0:5000; Ctrl+C = clean stop
C: python3 client.py --host 192.168.75.10                   # interactive: balance A1001, deposit A1001 100 ...
C: python3 client.py --host 192.168.75.10 balance A1001     # one-shot, prints JSON, exit code 0/1
C: python3 demo/concurrent_withdrawals.py --host 192.168.75.10 --clients 50 --amount 30
C: python3 demo/multi_clients.py --host 192.168.75.10 --clients 8 --ops 25
C: python3 demo/error_cases.py --host 192.168.75.10
C: python3 demo/idempotency_demo.py --host 192.168.75.10
A: python3 server.py --max-clients 3       C: python3 demo/max_clients_demo.py --host 192.168.75.10 --limit 3
A: BANK_ENABLE_FAULT_INJECTION=1 python3 server.py
C: python3 demo/fault_injection_demo.py --host 192.168.75.10
C: python3 demo/durability_demo.py --host 192.168.75.10     # waits for Enter
A: kill -9 $(pgrep -f server.py); python3 server.py         # then press Enter on C
# ---- Bonus 1 (fresh bank.db* on A and B)
A: python3 server.py --node-id A --initial-role primary --peer 192.168.75.11:6000
B: python3 server.py --node-id B --initial-role backup  --peer 192.168.75.10:6000
C: python3 client.py --servers 192.168.75.10:5000,192.168.75.11:5000
C: python3 client.py --host 192.168.75.11 balance A1001     # backup -> NOT_PRIMARY
C: python3 demo/failover_demo.py --servers 192.168.75.10:5000,192.168.75.11:5000 --duration 40
A: kill -9 $(pgrep -f "server.py --node-id A")              # after ~10-15 s, while clients run
# ---- Bonus 2 (fresh bank.db* on A and B, rm -f witness_state.json on C)
C: python3 witness.py --port 7000
A: python3 server.py --node-id A --initial-role primary --peer 192.168.75.11:6000 --witness 192.168.75.20:7000
B: python3 server.py --node-id B --initial-role backup --peer 192.168.75.10:6000 --witness 192.168.75.20:7000
C: python3 witness.py --status 192.168.75.20:7000
C: python3 demo/split_brain_demo.py --servers 192.168.75.10:5000,192.168.75.11:5000 \
       --repl 192.168.75.10:6000,192.168.75.11:6000 --witness 127.0.0.1:7000
A: sudo bash demo/vm_partition.sh cut-link   then heal, cut-all, heal (show = list rules)
# ---- Tests and single-machine versions
python3 -m unittest -v tests/test_bank.py      # also tests/test_cluster.py, tests/test_witness.py
python3 demo/failover_demo.py --local;  python3 demo/split_brain_demo.py --local [--without-witness]
```